



Installation of AMSTer

(FOR Linux DISTROS)

Installation guide by:

*N. d'Oreye, D. Smittarello, M. Jaspard, G. Celli, Q. Glaude – ECGS
B. Smets – RMCA*

https://github.com/AMSTerUsers/AMSTer_Distribution

Version 7.0: Aug 20 2026

Preliminary information.....	2
0) Prepare the work.....	5
1) Installing some external components.....	6
2) Installing required tools	9
3) Installing AMSTer software	15
4) Directory structure for data, auxiliary data and results.....	21
5) Check installation	22
6) Operating with cron jobs	26
7) In case of insufficient RAM	27
8) Downloading the Sentinel-1 orbits	27
9) Updates in this manual.....	28

Preliminary information

AMSTer (Automated SAR & InSAR Mass processing Software for Multidimensional Time series) was initially named **MasTer**. Some publications refer to that former name.

AMSTer is composed of 3 main elements: **AMSTer toolbox**, **AMSTer Engine** and **MSBAS**.

AMSTer toolbox is a set of mostly shell scripts aiming at processing automatically a large number of interferometric pairs and feeding and running the **MSBAS** processor [Samsonov and d'Oreye, 2012, 2017; Samsonov et al., 2017, 2020] in order to obtain the desired deformation maps and time series in vertical and horizontal components. Geocoded amplitude, coherence, interferometric phase and deformation maps are computed using the **AMSTer Engine**, a command line InSAR processor derived from the Centre Spatial de Liege (CSL) InSAR Suite (CIS) [Derauw, 1999; Derauw et al, 2019].

This guide explains the installation procedure for **AMSTer (AMSTer toolbox, AMSTer Engine, MSBAS)**, on a Linux computer. It does not replace the [manual](#) file provided with the software distribution ([AMSTer_Manual_x.x.docx](#)), but rather provide much more details to allow any user with less-advanced knowledge of the Linux environment to properly install the software suite.

The installation procedure has been last tested with the **AMSTerEngine_v20260813** package, msbasv3 (20190121), msbasv4 (20201009), msbasv10 (20230601) and shell scripts dated from August 2026.

Sources and codes are licensed under GNU Affero General Public (AGPL) License¹ and freely available for noncommercial use from Github:

https://github.com/AMSTerUsers/AMSTer_Distribution

If you are not granted the access to the repository or the discussions group, please send an email to ndo@ecgs.lu.

Download:

Either from the **distribution repository**:

- Only the installer script (**AMSTer_install.sh**) if you do not need/want to make a manual installation at https://github.com/AMSTerUsers/AMSTer_Distribution/tree/main/Installer (**recommended**), or
- the full package at https://github.com/AMSTerUsers/AMSTer_Distribution

Or from the **development (private) repositories**:

- the documentation and manuals from [ndoreye/DOCS](#)
- the scripts from [ndoreye/SCRIPTS_MT](#)
- the MSBAS sources tuned for AMSTer Toolbox from [ndoreye/MSBAS](#) (also available directly from S. Samsonov; see manual)
- the AMSTerEngine sources from [ndoreye/_Sources_AE](#) (also available directly from D. Derauw; see manual)

Note: Gitkraken is a convenient interface for Github but is not mandatory.

¹ as published by the Free Software Foundation, either version 3 of the License, or any later version. See the GNU Affero General Public License at <https://www.gnu.org/licenses/>.

This installation procedure has been tested on:

- **Ubuntu 24.0.4 LTS**
- **Ubuntu 22.0.4 LTS**

though former installations were operational.

It is assumed that the user has the administrator rights on the computer and can provide the root privilege when the “sudo” command is executed in the terminal.

In this installation guide, each command line that has to be written and executed in a terminal is written in *italic blue*. Files names are in *blue*. Paths are given in *green*.

Note:

A script exists to assist you making the installation (at least for Debian OS). See *.../SAR/AMSTer/SCRIPTS_MT/Installer/AMSTer_install.sh* (in distribution repository), or *.../SAR/AMSTer/SCRIPTS_MT/zz_Uutilities_MT/AMSTer_install.sh* (in development repository).

It is highly recommended to use its version 7.0 or above, which install python3.12 in the virtual environment as described here and download the sources for you from GitHub.

Read carefully all the text and questions displayed by the script.

For other type of Linux OS, you may need to adapt the script or proceed manually by adapting the procedure below.

Note that the *AMSTer_install.sh* installation script may be run several times (either to correct a problem during the installation or to perform an update of any element at any time). However, if some elements were installed manually (eg Gitkraken, Fiji or QGIS...), the script may not properly detect them as they may have been installed in an expected. This must not be a problem though.

To make the installation using the *AMSTer_install.sh* script (highly recommended**), refer to then user manual (chapter 0.5).**

The script will make for you all the installations described in sections 0 - 3 here after.

Kind reminder: The software is provided for free. To help us carrying on with the development of the tool, we kindly ask you to cite the **AMSTer Software** in publications that contains results produced by the toolbox thanks to at least the most recent references from below:

For AMSTer

- Derauw, D., N. d'Oreye, M. Jaspard, A. Caselli, and S. Samsonov (2020), Ongoing automated ground deformation monitoring of Domuyo - Laguna del Maule area (Argentina) using Sentinel-1 MSBAS time series: Methodology description and first observations for the period 2015-2020., *Journal of South American Earth Sciences*, 104, 102850, doi:10.1016/j.jsames.2020.102850.
- d'Oreye N., Derauw, D., S. Samsonov, M. Jaspard, and D. Smittarello (2021), MasTer: a full automatic multi-satellite InSAR mass processing tool for rapid incremental 2D ground deformation time series. *Proc. IEEE IGARSS21*, Brussels, July 2021.
- Smittarello, D., d'Oreye, N., Jaspard, M., Derauw, D., & Samsonov, S. (2022). Pair selection optimization for InSAR time series processing. *Journal of Geophysical Research: Solid Earth*, 127, e2021JB022825.
- d'Oreye N., Derauw, D., S. Samsonov, M. Jaspard, D. Smittarello, G. Celli (2023), AMSTer: SAR & InSAR Automated Mass processing Software for Multidimensional time series. *Instruction manual V6.0*, Nov. 2023. 237 pp.

For AMSTer Engine

- Derauw, D., (1999). Phasimétrie par Radar à Synthèse d'Ouverture; théorie et applications. PhD Dissertation, University of Liège, 141 pages.
- Libert, L., Derauw, D., d'Oreye, N., Barbier, C., Orban, A., (2017). Split-band interferometry-assisted phase unwrapping for the phase ambiguities correction. *Remote Sensing*, 9(9), 879.

For MSBAS

- Samsonov, S., d'Oreye, N., (2012). Multidimensional time-series analysis of ground deformation from multiple InSAR data sets applied to Virunga Volcanic Province. *Geophysical Journal International* 191(3), 1095–1108. doi: 10.1111/j.1365-246X.2012.05669.x.
- Samsonov, S., d'Oreye, N., (2017). Multidimensional Small Baseline Subset (MSBAS) for two-dimensional deformation analysis: Case study Mexico City. *Canadian Journal of Remote Sensing* 82(6), 1–12. doi: 10.1080/07038992.2017.1344926
- Samsonov S., W. Feng, A. Peltier, H. Geirsson, N. d'Oreye, KK. F.Tiampo (2017). Multidimensional Small Baseline Subset (MSBAS) for volcano monitoring in two dimensions: opportunities and challenges. Case study Piton de la Fournaise volcano. *Journal of Volcanology and Geothermal Research*, Vol 344, 121-138, <https://doi.org/10.1016/j.jvolgeores.2017.04.017>
- Samsonov, S., A. Dille, O. Dewitte, F. Kervyn, and N. d'Oreye (2020). Satellite interferometry for mapping surface deformation time series in one, two and three dimensions: A new method illustrated on a slow-moving landslide. *Engineering Geology*, (266), 105471, doi:10.1016/j.enggeo.2019.105471.items

For DORIS

- Kampes, B.M., Hanssen, R.F., Perski, Z., (2003). Radar interferometry with public domain tools. In: *Third International Workshop on ERS SAR Interferometry, FRINGE03*, Frascati, Italy, 1-5 Dec. 2003, 6 pages.

0) Prepare the work

The tools will need the following directories:

- /opt/local/bin
- \$HOME/SAR/EXEC
- \$HOME/SAR/AMSTer/DOC
- \$HOME/SAR/AMSTer/AMSTerEngine/_Sources_AE/Older
- \$HOME/SAR/AMSTer/MSBAS
- \$HOME/SAR/AMSTer/SCRIPTS_MT

Where \$HOME/ is obviously your home directory.

It is also advised to source your .barshrc in your **.bash_profile** as all the config will be set in **.bashrc**.

If you do not have these files, create them and ensure you can write and execute them.

If you have old AMSTer installation parameters in your **.bashrc**, it is advised to remove them and write new ones based on the instructions below.

1) Installing some external components

The following tools and apps are not all mandatory but experience shows that it may greatly help the user.

Note: if apt name or lib has changed, you can search by keyword the new name by typing at the Terminal:

- `sudo apt-cache search <keyword>`

Update installation:

- If not done yet:
 - `sudo apt install make`
 - `sudo apt install gzip`
- Nothing can be compiled without make. The AMSTer_Distribution is obtained from GitHub with git or, failing that, from a zip archive, hence also:
 - `sudo apt install git`
 - `sudo apt install unzip`
- To ensure update version of OS, type at terminal:
 - `sudo apt-get update`
 - `sudo apt -y upgrade`
- For old version of Ubuntu (<18):
 - `sudo apt install snapd`

GitKraken :

Although not mandatory, Gitkraken is a useful tool to sync with the last versions of AMSTer Toolbox.

- Command line download ([gitkraken-amd64.deb](#)), and install by typing
 - `wget -c https://release.gitkraken.com/linux/gitkraken-amd64.deb`
 - `sudo apt install ./gitkraken-amd64.deb`
- In case of manual download ([gitkraken-amd64.deb](#) from web site), install by typing
 - `sudo dpkg -i gitkraken-amd64.deb`
- may also be done using snap:
 - `sudo snap install gitkraken --classic`
- Connect e.g. via [Github](#)
- Create profile in GitKraken
- Launch GitKraken
- Clone Repo > with URL :
 - Where to Clone: `$HOME/SAR/`
 - Repo to clone: URL to AMSTerSoftware_Distribution

To display raster figure: XnView, GIMP, nomacs or Graphic Converter

For instance, see <https://www.gimp.org/downloads/>

Or at Terminal :

- `sudo add-apt-repository ppa:ubuntuhandbook1/gimp`

- `sudo apt update`
- `sudo apt install gimp`

nomacs is a lighter alternative to open .ras files. Install it with:

- `sudo apt install nomacs`

Editor:

- Get a good text editor you like

FTP:

- Get a convenient ftp software that you like

gnu fortran (not mandatory, and no longer installed by the installer script):

- Get it from http://fortran-lang.org/learn/os_setup/install_gfortran
- install with:
 - `sudo apt-get install gfortran`

GMT and GDAL:

GMT, GDAL and some associated utilities are required for plots, images and GIS manipulations.

- A former python gdal installed system wide by apt is the usual cause of the "gdal_array" errors met later when building the python bindings. If you hit them, make a fresh cleaning first:
 - `sudo apt remove --purge libpython3.6* libpython3.7*`
 - `sudo apt remove --purge python3-gdal python3-numpy python3-scipy python3-pandas python3-matplotlib python3-rasterio`
 - `sudo apt autoremove --purge`
 - `sudo apt clean`
- Then, at Terminal, type:
 - `sudo apt install -y \`
`gdal-bin libgdal-dev \`
`libhdf5-dev libnetcdf-dev libopenjp2-7-dev \`
`proj-bin libproj-dev \`
`libgeos-dev \`
`build-essential python3.12 python3.12-venv python3.12-dev`

where python3.12-venv and python3.12-dev are needed to create the virtual environment and to compile some python packages (see below).

- `sudo apt install gmt`
- `sudo apt install -y graphicsmagick ffmpeg` (no capital letter, and do not frame the two names within quotes)
- `sudo apt install libmpich-dev`
- `sudo apt install libopenmpi-dev`

Note: openjpeg is not needed as such: libopenjp2-7-dev above already provides what gdal requires.

You can check your gdal version with

`gdalinfo --version`

To display geocoded products: QGIS

- Visit <https://qgis.org> and get it eg as follow:
- At Terminal type:
 - o `sudo apt install gnupg software-properties-common`
 - o `sudo mkdir -m755 -p /etc/apt/keyrings`
 - o `wget -qO- https://qgis.org/downloads/qgis-archive-keyring.gpg | sudo tee /etc/apt/keyrings/qgis-archive-keyring.gpg > /dev/null`
 - o `sudo tee /etc/apt/sources.list.d/qgis.sources <<-EOF`
Types: deb
URIs: <https://qgis.org/ubuntu>
Suites: \$(lsb_release -c -s)
Components: main
Signed-By: /etc/apt/keyrings/qgis-archive-keyring.gpg
EOF
 - o `sudo apt update`
 - o `sudo apt install qgis qgis-plugin-grass`
- note some interesting plugins are:
 - o [point sampling tool](#)
 - o [PointConnector](#)
 - o [Profile tool](#)
 - o [Qdraw](#)
 - o [QuickMapServices](#)
 - o [RasterDataPlotting](#) (may require to install python first)
 - o [Serval](#)
 - o [Temporal/Spectral Profile Tool](#)
 - o [Value Tool](#)

For special characters if you keyboard sometimes plays it funny:

- o Install Ubuntu package [Character Map](#)

2) Installing required tools

AMSTer needs some pre-existing software and libraries to work properly. This is what we call the “dependencies”. So, we first need to install them. These installations are explained here, sorted by type of installation procedure.

GNU functions and utilities:

Several commands might exist by default on Linux, but getting the gnu version again is wise.

- To be sure to have the latest version of gnu tools, type at the Terminal:

- `sudo apt install sed`
- `sudo apt install gawk`
- `sudo apt install coreutils` (i.e. for `gdate`, `gstat`)
- `sudo apt install findutils` (i.e. for `find`)
- `sudo apt install grep` (i.e. for `ggrep`)
- `sudo apt install wget` (needed i.e. to download the S1 orbits)
- `sudo snap install curl`

and add the path to `curl` (e.g. `/snap/bin`) in your `$PATH`

and add the directory that contains `curl` (e.g. `/snap/bin`) to your `PATH` in the `.bashrc`

check it with:

- `which curl`

- Check that all these gnu functions are stored in the `/usr/bin` directory. If not, move them there. That directory MUST be defined as `${PATHGNU}` state variable in your `.bashrc`. At Terminal, type

- `which sed`
- `which gawk`
- `which date`
- `which stat`
- `which find`
- `which grep`

If answer to all these lines is not the same as `echo ${PATHGNU}`, edit your `.bashrc` and change `${PATHGNU}` state variable. It must be something like

`export PATHGNU=/usr/bin`

Don't forget that a reboot (or `source .bashrc`) is required for changes in `.bashrc` to be taken into account.

- Just in case one of these functions would be called without its g-name in a script, it is recommended to have them in `${PATHGNU}` with both names, that is with and without heading `g` (that is **sed** and **gsed**, **awk** and **gawk**, **grep** and **ggrep**, **date** and **gdate**, **stat** and **gstat**). This can be simply done with a link. In Terminal, type:

- `cd ${PATHGNU}`
- `sudo ln -s sed gsed`
- `sudo ln -s gawk awk` (beware if `awk` already exist and would be a link pointing toward another version of `awk`. Replace it with our `gawk`)

Note: `awk` is not always installed by default on Linux, which is why `gawk` is installed above and linked to both names. If `gawk` is not in `${PATHGNU}`, link it from where it really is:

- `sudo ln -sf "$(which gawk)" ${PATHGNU}/gawk`
- `sudo ln -sf "$(which gawk)" ${PATHGNU}/awk`
- `sudo ln -s date gdate`
- `sudo ln -s stat gstat`

- `sudo ln -s find gfind`

And to be sure...(I know it is overkill, but one never know...):

- `sudo ln -s grep ggrep`
- `sudo ln -s seq gseq`
- `sudo ln -s uniq guniq`
- `sudo ln -s readlink greadlink`
- `sudo ln -s xargs gxargs`
- `sudo ln -s du gdu`

Check also that wget is in `$(PATHGNU)` with

- `which wget.`

If it is not, link it there:

- `sudo ln -sf "$(which wget)" $(PATHGNU)/wget`

Libraries & modules:

- Install libraries and modules :
 - `sudo apt install clang-18` (or clang-20; version 18 or above is mandatory from AMSTerEngine_20250805)
- If the clang package of your distribution is too old, download and run the LLVM script instead:
 - `wget https://apt.llvm.org/llvm.sh`
 - `chmod +x llvm.sh`
 - `sudo ./llvm.sh 20`
- Then install the other libraries:
 - `sudo apt install libfftw3-dev`
 - `sudo apt install libfftw3-long3`
 - `sudo apt install libfftw3-single3`
 - `sudo apt install libgeotiff-dev`
 - `sudo apt install libtiff-dev`
 - `sudo apt install libxml2`
 - `sudo apt install libxml2-dev`
 - `sudo apt install liblapack-dev`
 - `sudo apt install libomp-dev`
 - `sudo apt install libopenblas-dev`
 - `sudo apt install g++`
 - `sudo apt install -y espeak` (optional; to make your computer talking during mass processing)
 - `sudo apt install parallel`
 - `sudo apt install mpich`
 - `sudo apt install libgsl-dev`
 - `sudo apt install ghostscript` (used by GraphicsMagick for EPS/PS/PDF)
 - `sudo apt install gsfonts` (fonts used by GraphicsMagick)

Imagemagick is needed e.g. for the convert function. It must be installed with the image-magick-6-common module. Another option is to use **graphicsmagick**, which is more performant than imagemagick but less compatible. It has also the advantage that it does not need a `policy.xml` file. This last file is used by Imagemagick to define the permissions to read/write EPS, PS, PDF files... and allocate a maximum size to the created files (height, width..). Graphicsmagick does not restrict the sizes, but permission to read/write EPS, PS, PDF files... may be managed by external utilities (Ghostscript).

Do not install both to avoid conflict, or manage the priority with update-alternatives (e.g. 'sudo update-alternatives --config convert' will let you pick whether /usr/bin/convert points to ImageMagick or GraphicsMagick).

Hence, choose between

- `sudo apt install graphicsmagick`
- `sudo apt install graphicsmagick-imagemagick-compat` # allows usage in script

Note: the installer script installs graphicsmagick and its compatibility layer, i.e. the two lines above. With that choice there is no policy.xml at all, hence no restriction on the height and width, and the permissions to read/write EPS, PS and PDF are managed by Ghostscript installed above: the whole paragraph below about policy.xml then does not apply.

and

- `sudo apt install imagemagick`
- `sudo apt install imagemagick-6-common`

Attention: with imagemagick, for `convert` function to operate properly **ps** and **eps** files, change permissions in `/etc/ImageMagick/policy.xml` (or `/etc/ImageMagick-6/policy.xml`) and set (e.g. using nano as sudo):

```
</policy domain="coder" rights="none" pattern="PS" />
```

as

```
</policy domain="coder" rights="read|write" pattern="PS" />
```

(i.e. with pipe between read and write), and

```
</policy domain="coder" rights="none" pattern="EPS" />
```

as

```
</policy domain="coder" rights="read|write" pattern="EPS" />
```

Also, change as follow:

```
</policy domain="resource" name="height" value="16KP" />
```

as

```
</policy domain="resource" name=" height" value="32KP" />
```

and

```
</policy domain="resource" name="width" value="16KP" />
```

as

```
</policy domain="resource" name=" width" value="32KP" />
```

and

```
</policy domain="resource" name="disk" value="1GiB" />
```

as

```
</policy domain="resource" name="disk" value="8GiB" />
```

For some plotting applications: Java (preferably installed before Fiji)

Install it with :

- `sudo apt install default-jdk`

or manually :

- see <http://www.java.com>
- follow instructions to install Java

Notes:

- if some scripts crash with the following message, re-install java and then re-install Fiji:
The operation couldn't be completed. Unable to locate a Java Runtime that supports (null).
 - One may need to define `JAVA_HOME` state variable in the `.bashrc` for some application using it. Check where it is installed by typing:
 - `java -XshowSettings:properties -version 2>&1 > /dev/null | grep 'java.home'`
 and update the `.bashrc` by adding a line with the answer of the line above:
 - `export JAVA_HOME=PATH_FROM_LINE_ABOVE`
- If that command does not answer anything usable, you can also try:

- o `sudo update-alternatives --config java`

and note the path to the desired version of java, dropping everything from `/bin` on, e.g. `/usr/lib/jvm/java-11-openjdk-amd64`

Fiji/ImageJ:

Do not install it with the terminal !!! You must use the stand-alone version.

- Download latest version from <https://imagej.net/software/fiji/downloads>
- Unzip the compressed file in `$HOME/SAR/EXEC`.
- Locate `ImageJ-linux64` in your `$HOME/SAR/EXEC/Fiji.app` directory.
- Update `$PATHFIJI` in your `.bashrc`. It must be something like
`export PATHFIJI=$HOME/SAR/EXEC/Fiji.app/`

Note: Fiji changed its launcher in February 2025. Before that date the executable is named `ImageJ-linux64`, which is what the installer script searches for; from February 2025 (Jaunch launcher) it is simply named `fiji`. If your installation only contains the latter, set `$PATHFIJI` to the directory that contains it and check that the scripts calling Fiji do find it.

- Store the installed source e.g. in `$HOME/SAR/EXEC/Sources_Installed`.

snaphu:

- Download latest version (e.g. `snaphu-v2.0.6.tar`) from <https://web.stanford.edu/group/radar/softwareandlinks/sw/snaphu/> and store it in `$HOME/SAR/EXEC`.
- unzip it e.g. using `tar -zxvf`
- edit the Makefile in the `/src` directory to specify your compiler and the optimization flags. The default `CFLAGS` line of snaphu (since v2.0.7) forces `-arch x86_64`, which is not compatible with every architecture, and it must have no static option. The installer script rewrites that line with:
`sed -i 's/^CFLAGS.*CFLAGS = $(OPTIMFLAGS) -Wall/' Makefile`
- compile: at Terminal in `/src` dir, type `make`
(it seems that you can ignore warnings; most recent compilers are more sensitive)
- Move compiled `snaphu` from `/bin` into `$HOME/SAR/EXEC`
- Download also the `snaphu_man1.txt` if you need more info about snaphu and its configurations.
- Store the installed source e.g. in `$HOME/SAR/EXEC/Sources_Installed`.

Note: this installation can also be made using `install`, but it might not be the latest version and path must be checked...

- o `sudo apt install snaphu`

cpxfiddle:

Not mandatory, but pretty convenient for creating raster files for quick looking at results.

- Download source from https://github.com/TUDeftGeodesy/Doris/tree/master/sar_tools
- Cut/paste source in new document with your favorite editor
- Save the file in `$HOME/SAR/EXEC/cpxfiddle.cc`
- Edit `$HOME/SAR/EXEC/cpxfiddle.cc` and replace the line
`if (argv[optind]=='\0') (around line 1857)`
by
`if (argv[optind]==0 || argv[optind][0]=='\0')`

If the compilation still fails because of the `register` keyword (just a compiler hint, ignored by real compilers for decades), delete that keyword in the three spots where it appears (lines 256, 257 and 635).

- In Terminal:
 - o `cd $HOME/SAR/EXEC`
 - o `make -n cpxfiddle`
 - o `g++ -O -c -ocpxfiddle.o cpxfiddle.cc`
 - o `g++ -O cpxfiddle.o -o cpxfiddle`
- Clean sources in `$HOME/SAR/EXEC/cpxfiddle.o`
- Store the installed source `cpxfiddle.cc` e.g. in `$HOME/SAR/EXEC/Sources_Installed`
- Update `$PATHTOCPXFIDDLE` in your `.bashrc`. It must be something like
`export PATHTOCPXFIDDLE=${HOME}/SAR/EXEC/`

gnuplot: gnuplot is used e.g. to create time series figures or baseline plots.

- At Terminal:
 - o `sudo apt install gnuplot-x11`
 - o `which gnuplot`
- Ensure that the path provided by the former command displays the same path as `$PATHGNU` in your `.bashrc`, that is `/usr/bin`

python:

- At Terminal:
 Create a virtual environment. python3 is used, that is python 3.12 on Ubuntu 24.04, installed with the gdal libraries in section 1. Start from a clean state:

o `rm -rf /opt/local/amster_python_env/`

and, if you already met "gdal_array" errors, clean the pip caches as well:

o `rm -rf ~/.cache/pip`

then create the environment:

- o `sudo mkdir -p /opt/local`
- o `sudo chown -R $USER /opt/local`
- o `python3 -m venv /opt/local/amster_python_env`
- o `sudo chown -R $USER /opt/local/amster_python_env`
- o `source /opt/local/amster_python_env/bin/activate`
- o `/opt/local/amster_python_env/bin/pip install --upgrade pip`

numpy and the gdal python bindings (osgeo) must be installed FIRST and separately, before the requirements file: the bindings are compiled against the gdal really installed on your system, hence they must match its version exactly. Do NOT put gdal in requirements.txt.

- o `pip install numpy>1.0.0 wheel setuptools>=67`
- o `pip install gdal[numpy]=="$(gdal-config --version).*"`

Load the remaining modules by creating a requirements.txt and installing it:

- o `echo "scipy==1.11.4" > /opt/local/requirements.txt`
- o `echo "matplotlib==3.8.2" >> /opt/local/requirements.txt`
- o `echo "shapely==2.0.2" >> /opt/local/requirements.txt`
- o `echo "utm==0.7.0" >> /opt/local/requirements.txt`
- o `echo "pyproj" >> /opt/local/requirements.txt`
- o `echo "PyQt6==6.7.1" >> /opt/local/requirements.txt`
- o `echo "networkx==3.2.1" >> /opt/local/requirements.txt`
- o `echo "geopandas==0.14.3" >> /opt/local/requirements.txt`
- o `echo "scikit-gstat==1.0.18" >> /opt/local/requirements.txt`
- o `echo "rasterio==1.3.9" >> /opt/local/requirements.txt`
- o `echo "pandas==2.2.1" >> /opt/local/requirements.txt`
- o `echo "glob2==0.7" >> /opt/local/requirements.txt`
- o `echo "opencv-contrib-python==4.8.1.78" >> /opt/local/requirements.txt`

Note that opencv-python is not listed: it is already included in opencv-contrib-python.

Then install everything in the virtual environment:

- `/opt/local/amster_python_env/bin/pip install -r /opt/local/requirements.txt`

To ensure using the here installed Python3 version and modules in the virtual environment, all the python scripts must start with the line

```
#!/opt/local/amster_python_env/bin/python
```

Note: Ubuntu 24.04 ships python 3.12, with which the osgeo/gdal bindings may still fail to build. If they do, fall back on python 3.10 from the deadsnakes repository and recreate the environment with it:

- `sudo apt install -y software-properties-common`
- `sudo add-apt-repository -y ppa:deadsnakes/ppa`
- `sudo apt update`
- `sudo apt install -y python3.10 python3.10-venv python3.10-dev`
- `sudo apt remove --purge -y python3-gdal`
- `rm -rf /opt/local/amster_python_env`
- `python3.10 -m venv /opt/local/amster_python_env`

then repeat the steps above from the pip upgrade on.

x-terminal-emulator: A terminal emulator is required to open terminal from command line (e.g. when splitting mass processing).

- Install it by typing at the terminal:
 - `sudo apt install deepin-terminal`

It must be able to open on your display. Check which DISPLAY your session runs on with the command below; if several are listed, choose the one you are working on (something like :0.0) and export it before launching the scripts that split the mass processing:

- `who -m | cut -d "(" -f 2 | cut -d ")" -f 1`

3) Installing AMSTer software

AMSTer consists of several packages that need to be obtained or downloaded from different sources:

- **AMSTerEngine** (core parts: InSAR and MSBASTools):
 - See **Preliminary information/Sources and codes** above for Github instructions, or
 - contact: dderauw@ecgs.lu
 - **AMSTer Toolbox (SCRIPTS_MT: Main Toolbox mass processing scripts)**:
 - See **Preliminary information/Sources and codes** above for Github instructions, or
 - contact ndo@ecgs.lu
 - **MSBAS** (2D or 3D inversion time series):
 - See **Preliminary information/Sources and codes** above for Github instructions, or
 - Download original version from: <https://doi.org/10.4095/313749>, or
 - Request code tuned for AMSTer with a make file adapted to Mac and Linux, contact ndo@ecgs.lu
- In case of problem, contact sergey.samsonov@canada.ca

All these components are gathered in the AMSTer_Distribution repository. The installer script offers to download (or update) it for you in `$HOME/SAR/EXEC/AMSTer_Distribution`, that is where it keeps all the sources it installs from. To do it manually, either clone it, which is recommended since it can then be updated later with a simple git pull:

- `mkdir -p $HOME/SAR/EXEC`
- `cd $HOME/SAR/EXEC`
- `git clone --depth 1 --branch main`
https://github.com/AMSTerUsers/AMSTer_Distribution.git

or download the zip archive of the branch and decompress it. Beware that the archive expands into a directory named AMSTer_Distribution-main, which must be renamed:

- `wget -O AMSTer_Distribution-main.zip`
https://github.com/AMSTerUsers/AMSTer_Distribution/archive/refs/heads/main.zip
- `unzip -o AMSTer_Distribution-main.zip -x "__MACOSX/*"`
- `mv -f AMSTer_Distribution-main AMSTer_Distribution`

3.1 Compile **AMSTerEngine**:

If executable files were provided (zip file),

- simply unzip it and copy the binaries in `$HOME/SAR/AMSTer/AMSTerEngine`
- then store the zipped file in `$HOME/SAR/AMSTer/AMSTerEngine/_Sources_AE/Older/Vmostrecent_AMSTerEngine` where *mostrecent* is the date of the last version, which is in the name of the zip file. This date must be as YYYYMMDD and will be used by some scripts to track (and log) the version used during processing.
- Update `$PATH` in your `.bashrc`. It must be something like `PATH=$PATH:$HOME/SAR/AMSTer/AMSTerEngine`

- If you were provided with the source code, it comes as a tar.xz archive named **AMSTerEngineYYYYMMDD.tar.xz**, where **YYYYMMDD** is the date of the version. That date is used by some scripts to track (and log) the version used during processing. Store the archive in **\$HOME/SAR/AMSTer/AMSTerEngine/_Sources_AE/Older/VYYYYMMDD_AMSTerEngine/**. Beware: the path to the sources must not contain any white space, otherwise the compilation fails.
- Then, in a Terminal, decompress the archive.
Note: If these archives were created on macOS and carry PAX extended headers that GNU tar does not know; the `--warning=no-unknown-keyword` option simply silences the resulting harmless messages:

```
cd $HOME/SAR/AMSTer/AMSTerEngine/_Sources_AE/Older/VYYYYMMDD_AMSTerEngine
tar --warning=no-unknown-keyword -xf AMSTerEngineYYYYMMDD.tar.xz
```


The recent distributions contain a second archive in a sub directory named Archives. If that directory exists, decompress that archive as well:

```
cd AMSTerEngineYYYYMMDD/Archives
tar --warning=no-unknown-keyword -xf Mas*.tar.xz
```
- Compile the InSAR part.
USEOPENMP=YES enables the parallelisation and is recommended (it requires libomp-dev, installed in section 2); without it, simply drop that option:

```
cd InSAR/sources
make USEOPENMP=YES
```
- Copy the history file and move the binaries in **\$HOME/SAR/AMSTer/AMSTerEngine**. The bin directory must contain initInSAR; if it does not, you are not in the right directory:

```
cp _History.txt $HOME/SAR/AMSTer/AMSTerEngine/
mv -f ../bin/* $HOME/SAR/AMSTer/AMSTerEngine/
```
- Compile the MSBASTools the same way. Here the bin directory must contain getLineThroughStack:

```
cd ../../MSBASTools/sources
make USEOPENMP=YES
mv -f ../bin/* $HOME/SAR/AMSTer/AMSTerEngine/
```
- Clean (delete) the decompressed directories; keep only the tar.xz archive in **\$HOME/SAR/AMSTer/AMSTerEngine/_Sources_AE/Older/VYYYYMMDD_AMSTerEngine**.
- Update **\$PATH** in your .bashrc. It must be something like `PATH=$PATH:$HOME/SAR/AMSTer/AMSTerEngine`

Note that if you are provided with an updated version of AMSTer Engine (source code), you can simply update it by copying the zipped file in **\$HOME/SAR/EXEC** and run the script **/\$HOME/SAR/AMSTer/SCRIPTS_MT/zz_Uilities_MT_Ndo/UpdateAMSTerEngine.sh** with the path to the zipped sources file as parameter.

3.2 Compile **MSBAS**:

- The MSBAS sources are provided as zip files in the AMSTer_Distribution, in **MSBAS_sources/msbasvNN/**. Several versions exist, e.g.
 - **msbas_20201009_wExtract_Unified_20220919_Optimized_v1.2.2_Gilles.zip** for msbasv4 (2D) running in parallel on all the available cores,
 - **msbas_20201009_wExtract_Unified_20220919_Optimized_v1.3.2_Full3D.zip** for the full 3D decomposition, or

- o msbas_v10_20230601_Gilles.zip for msbasv10.

Copy the chosen one where you want to keep the installed sources, e.g. in `$HOME/SAR/EXEC/Sources_Installed/Former_msbas/`, then decompress it:

```
cd $HOME/SAR/EXEC/Sources_Installed/Former_msbas
unzip -o msbas_..._Optimized_vX.Y.Z_Name.zip -x "__MACOSX/"
```

- Enter the directory really created by unzip: it is not necessarily named after the zip file. A zip renamed e.g. msbas_..._V1_1.zip (dots turned into underscores by a mail server or a copy on a FAT disk) still expands into msbas_..._V1.1/. List it with:

```
ls -d msbas_*/
cd msbas_..._Optimized_vX.Y.Z_Name
```

- These archives provide several makefiles, some flavoured for Linux and some hardcoding the MacPorts or Homebrew paths of a Mac. Copy the one you need in the list below over Makefile (do not delete the current Makefile before having checked that its replacement is there, as some distributions provide only part of them):

- o Makefile_clang21_cpp17,
- o msbasv10_Makefile_port,
- o msbasv10_Makefile_brew,
- o Makefile_brew_cpp17,
- o Makefile_clang14_cpp11, and finally the plain
- o Makefile

```
cp -f Makefile_clang21_cpp17 Makefile
```

- Delete any msbas binary shipped within the zip or left by a former build before compiling: the msbasv4 distribution ships a prebuilt x86_64 binary and a failed compilation would leave it in place, where it could be taken for a successful build:

```
rm -f msbasv4 msbasv4_3D msbasv10 msbasv10_3D msbas
```

- then compile and copy the executable really produced in `$HOME/SAR/AMSTer/MSBAS` (it seems that you can ignore warnings; most recent compilers are more sensitive):

```
make all
cp -f msbasvNN $HOME/SAR/AMSTer/MSBAS/
```

- msbas_extract is only provided with msbasv4 and msbasv4_3D (its sources are identical in both). With these versions, compile it as well, with the Linux flavoured makefile:

```
cd msbas_extract
cp -f msbas_extract_Makefile_port Makefile
rm -f msbas_extract
make
cp -f msbas_extract $HOME/SAR/AMSTer/MSBAS/
```

If the compilation fails, check first the dependencies, which differ between the msbas generations: msbasv4 and v4_3D need `g++ + libgdal-dev + liblapack-dev + libomp-dev`, while msbasv10 and v10_3D need `mpich + libgdal-dev + liblapack-dev`.

- Former versions of msbas already present in `$HOME/SAR/AMSTer/MSBAS` should be kept as a backup in `$HOME/SAR/EXEC/Sources_Installed/Former_msbas/` before installing the new one, and the sources must be kept there as well (only remove the `__MACOSX`, `.DS_Store` and `*.o` leftovers).

- Update `$PATH` in your `.bashrc`. It must be something like
`PATH=$PATH:$HOME/SAR/AMSTer/MSBAS`

Note:

The version `msbas_20201009_wExtract_Unified_20220919_Optimized_v1.3_Full3D.zip` is dedicated to very specific usage, that is when enough diversity in viewing geometries is

achieved to perform full 3D inversion. Usually, this can only be achieved when data are acquired along Ascending and Descending orbits by sensors with right and left looking capabilities. See AMSTer manual.

3.3 Get the **AMSTer Toolbox** scripts at the right place:

- Ensure that the scripts (obtained from Github or from unzipped file) are stored at the right place:
`$HOME/SAR/AMSTer/SCRIPTS_MT`
- Update `$PATH` in your `.bashrc`. It must be something like
`PATH=$PATH:$HOME/SAR/AMSTer/SCRIPTS_MT`
`PATH=$PATH:$HOME/SAR/AMSTer/SCRIPTS_MT/zz_Uutilities_MT`
`PATH=$PATH:$HOME/SAR/AMSTer/SCRIPTS_MT/zz_Uutilities_MT_NdO`
`PATH=$PATH:$HOME/SAR/AMSTer/SCRIPTS_MT/_cron_scripts`
`PATH=$PATH:$HOME/SAR/AMSTer/SCRIPTS_MT/optimtoolbox`
`PATH=$PATH:$HOME/SAR/AMSTer/SCRIPTS_MT/diagtoolbox`
`PATH=$PATH:$HOME/SAR/AMSTer/SCRIPTS_MT/AMSTerOrganizer`
- The order matters: `${PATHGNU}` (i.e. `/usr/bin`) and `/opt/local/bin` must come before the other default directories, so that the gnu commands and the python of the virtual environment have priority, while the `SCRIPTS_MT`, `AMSTerEngine`, `MSBAS`, `EXEC` and `curl` (e.g. `/snap/bin`) directories are appended after. If you use TigerVNC, `/usr/local/tigervnc/bin/` is appended there as well.
- If scripts were receiver from a zipped file, it can be stored in
`$HOME/SAR/AMSTer/EXEC/Sources_Installed`

3.4 Define the environmental variables:

AMSTerEngine, MSBAS and AMSTer Toolbox requires environmental variables that will be called from the scripts. These are:

`PATH_SCRIPTS=${HOME}/SAR/AMSTer` : path to where is located the directory `SCRIPTS_MT`
(i.e. the directory containing all the scripts)

`PATHGNU=/usr/bin` : path to where the `gnu utilities` are stored
(**Note:** in case you have several versions of the gnu utilities, it is highly recommended to have the path to `/usr/bin` **defined first** in your `$PATH`)

`PATHFIJI=$HOME/SAR/EXEC/Fiji.app/` : path to where `Fiji` application is

`PATHCONV=/usr/bin` : path to where the `convert` command is (from `ImageMagick`)

`PATHTOCPXFIDDLE=${HOME}/SAR/EXEC/` : path to where the `cpxfiddle` command is (from TU Delft)

`JAVA_HOME` may also be required by some applications. Get its value with
`java -XshowSettings:properties -version 2>&1 > /dev/null | grep 'java.home'`
 or with

`sudo update-alternatives --config java`

and export it in your `.bashrc` (the path must be quoted if it contains white spaces).

OPENBLAS_NUM_THREADS=1 must also be exported. This is the number of threads used for the parallel computation and it is required by the optimized compilation of msbas; a higher value makes msbas crash. Keep it in mind if one day you run another parallelised application that could handle more.

The path to some disks where data, intermediate results and final results will be stored must also be defined by state variable.

At least 5 variables **must be defined**, that is:

PATH_1650, PATH_3600, PATH_3601, PATH_3602 and PATH_DataSAR.

More can be defined, e.g.: PATH_1660, PATH_3610, PATH_3611, PATH_3612, PATH_SynoData and PATH_HOMEDATA, which are optional and used e.g. to spread the computation over more disks (see e.g. **__SplitSession.sh**).

Names are inherited from architecture at ECGS and will be called throughout the processing. For security reason, it is a good practice to have them pointing to different physical drives, if possible, but this is not mandatory; they can also point toward directories instead of disks, e.g.:

PATH_1650=/Volumes/hp-1650-Data_Share1	(disk or dir where data or results will be stored)
PATH_3600=/Volumes /hp-D3600-Data_Share1	(disk or dir where data or results will be stored)
PATH_3601=/Volumes /hp-D3601-Data_RAID6	(disk or dir where data or results will be stored)
PATH_3602=/Volumes /hp-D3602-Data_RAID5	(disk or dir where data or results will be stored)
PATH_DataSAR=/Volumes /DataSAR/	(disk or dir where auxiliary data will be
	automatically stored such as orbits, geoid etc...
	in a dir named SAR_AUX_FILES)

Note also that more disks can be defined and used e.g. for sharing the computation through several disks when intensive computation is required.

When processing ENVISAT or Sentinel 1 data, AMSTerEngine will require orbital information. It will usually pick them for you and store them automatically in dedicated directories where it will get them when required. These directories **must be defined** by the following state variable names:

S1_ORBITS_DIR=\${PATH_DataSAR}/SAR_AUX_FILES/ORBITS/S1_ORB
 ENVISAT_PRECISES_ORBITS_DIR=\${PATH_DataSAR}/SAR_AUX_FILES/ORBITS/ENV_ORB
 (the vor_gdr_d sub directory must exist inside that directory)

To create DEM corrected from geoid height, the following state variable must be defined:

EARTH_GRAVITATIONAL_MODELS_DIR=\${PATH_DataSAR}/SAR_AUX_FILES/EGM

A directory named EGM96 must exist in that directory and it must contain the geoid file WW15MGH.DAC. It can be downloaded from (link checked 05 10 2022):

<https://web.archive.org/web/20130314064801/http://earth-info.nga.mil/GandG/wgs84/gravitymod/egm96/binary/WW15MGH.DAC>

To create DEM, EXTERNAL_DEMS_DIR state variable needs to be defined in your .bashrc (e.g.: EXTERNAL_DEMS_DIR=/YourPath/DEMS/).

When you create your DEM (e.g. using **getSRTMDEM** function since AMSTerEngine V20241010– see manual), it is strongly advised to move (and rename if needed) the created DEM to a directory from where you will call it in the **ParametersFile.txt** files. By default, the DEM will have the same name than the given kml.

You can also create automatic water body mask using **getSRTMDEM** function since AMSTerEngine V20241010 (see manual). For that, you will need to have a state variable EXTERNAL_MASKS_DIR.

If an EXTERNAL_MASKS_DIR environmental variable is defined in your .bashrc (e.g.: EXTERNAL_MASKS_DIR=/YourPath/MASKS/), the waterBody mask will be saved in that EXTERNAL_MASKS_DIR. In that case, it is also **strongly advised to move (and rename if needed)** the created mask to a directory from where you will call it in the **ParametersFile.txt** files.

Finally, the following directories must exist in `${PATH_DataSAR}`; create them with:

```
mkdir -p ${PATH_DataSAR}/SAR_AUX_FILES/DEM
mkdir -p ${PATH_DataSAR}/SAR_AUX_FILES/EGM/EGM96
mkdir -p ${PATH_DataSAR}/SAR_AUX_FILES/ORBITS/ENV_ORB/vor_gdr_d
mkdir -p ${PATH_DataSAR}/SAR_AUX_FILES/ORBITS/ERS
mkdir -p ${PATH_DataSAR}/SAR_AUX_FILES/ORBITS/S1_ORB/AUX_POEORB
mkdir -p ${PATH_DataSAR}/SAR_AUX_FILES/ORBITS/S1_ORB/AUX_RESORB
```

and download the geoid file WW15MGH.DAC with `wget` in

`SAR_AUX_FILES/EGM/EGM96/` (see section 5 and the link above).

Create an empty and properly protected `.netrc` in your home directory if you do not have one yet:

```
touch .netrc
chmod 0600 .netrc
```

because it is needed for the S1 orbits, the SRTM DEM creation, etc. See section 8 for its content.

Two commented lines are also added at the end of the `.bashrc`, to be uncommented and adapted if one day you need to limit the hardware usage while running `msbas` parallelised with OpenMP:

```
#export OMP_NUM_THREADS=10,8,4 and #export OMP_NUM_THREADS=4,3,2
```

4) Directory structure for data, auxiliary data and results

In addition to the structure of the installation directories, it is important to store your data, auxiliary data (DEM, masks, kml, orbits, geoid, parameters files...), intermediate results and final results in a structure as described below.

AMSTer expects a directory structure similar as what is shown below. Most of the directories where the intermediate and final results are stored will be created by the scripts based on the information provided in the [LaunchMTparameters.txt](#) file.

For those which must be created by the user (in full or based on name provided in the [LaunchMTparameters.txt](#) file), it is required to follow the following architecture and directories naming.

In *italic green* is the information that must be adapted to your satellite(s), orbital mode(s) and region(s). These directories can be located in the disk(s) you want.

Note that, for safety reasons, it is advised to store e.g. raw data and data in csl format (i.e. from AMSTerEngine) on different hard drives, or the mass processed results and the msbas results on different hard drives etc....

Names in **red** are mandatory.

Disk 1 (i.e. **PATH_1650**):

PATH_1650/kml	: where to store kml for cropping, or for coherence check
PATH_1650/Param_files	: where to store your LaunchMTparameters.txt file by <i>/SAT/TRK....</i>
PATH_1650/ SAR_CSL / <i>SAT/TRK/NoCrop</i>	: where data in csl format are stored (incl. crops)
PATH_1650/SAR_SM/AMPLITUDES/ <i>SAT/TRK/Region</i>	: where amplitudes gif will be computed
PATH_1650/ SAR_SM /MSBAS/ <i>Region</i>	: where compatible pairs for msbas are computed
PATH_1650/ SAR_SM /RESAMPLED/ <i>SAT/TRK</i>	: where data resampled on Global Primary are saved

Disk 2 (i.e. **PATH_3600**):

PATH_3600/**SAR_DATA**/*SAT/...* : where raw data are stored (safer to keep them on a different hard drive than 1650)

Disk 3 (i.e. **PATH_3601**):

PATH_3601/ SAR_MASSPROCESS / <i>SAT/TRK/Crop</i>	: where directories with each pair computation details and results are stored
PATH_3601/ SAR_MASSPROCESS / <i>SAT/TRK/Crop/Geocoded</i>	: where geocoded maps are stored in sub dir Amp, Coh, Defo etc...
PATH_3601/ SAR_MASSPROCESS / <i>SAT/TRK/Crop/GeocodedRasters</i>	: where rasters of geocoded maps (for quick look) are stored

Disk 4 (i.e. **PATH_3602**):

PATH_3602/**MSBAS_RESULTS**/*Region* : where msbas series are computed

Maybe in one of these disks, or somewhere else where enough storage room is available:

\$PATH_DataSAR/SAR_AUX_FILES/DEM	: where DEMs are stored
\$PATH_DataSAR/SAR_AUX_FILES/EGM96	: where geoid is stored
\$PATH_DataSAR/SAR_AUX_FILES/MASKS / <i>SAT/TRK</i>	: where coherence masks are computed
\$PATH_DataSAR/SAR_AUX_FILES/ORBITS / <i>SAT</i>	: where orbits are stored

Note that **\$PATH_DataSAR** is the state variable defined in .bashrc !!

Note: In Linux, to mount external drive, you must

- 1) Define the mounting point of your disks, e.g for a disk named "syno_sar":

```
sudo mkdir /mnt/syno_sar
sudo chown yourlogin /mnt/syno_sar
```

- 2) Add them in the [/etc/fstab](#). Example below shows the mounting of a Windows server and a Synology server:

```
# EG FOR A WINDOWS STORAGE SERVER
# IMPORTANT: Leave "cache=none" to avoid issues with 10Gbit Ethernet-Network on eno1
//hp-storeeasy.ecgs.welter/hp-1650-Data_Share1 /mnt/1650 cifs comment=systemd.au-
tomount,username=YOURUSERNAME,password=YOURPWD,uid=YOURID,gid=users,noatime,iocchar-
set=utf8,links,cache=strict,vers=3.1.1,soft,noserverino,nobrl,nounix 0 0

# EG FOR SYNOLOGY STORAGE
//syno-sar.ecgs.welter/DataSAR /mnt/syno_sar cifs comment=systemd.au-
tomount,username=YOURUSERNAME,password=YOURPWD,uid=YOURID,gid=1001,iocchar-
set=utf8,file_mode=0777,dir_mode=0777,noatime,rw,mfsymlinks,soft,nounix 0 0
```

5) Check installation

5.1 The directory structure

At the end, you should have AMSTer installed in a folder named “SAR” as shown in **Fig. 1**.

```

/$HOME/SAR/EXEC
|--Executables (cpxfiddle, snaphu...)
|--Sources_Installed

/$HOME/SAR/AMSTer
|--AMSTerEngine
|   |--_Sources_AE/Older
|   |   |--[several sources of old versions]
|   |   |--V20250808_AMSTerEngine
|   |--Executables...
|--DOC
|   |--AMSTerEngine
|   |--AMSTer_Toolbox
|   |--Biblio
|   |--LOGOS
|   |--MSBAS
|--MSBAS
|   |--3Dsbas_Archives
|   |   |--msbasv4
|   |--Executables...
|--SCRIPTS_MT
|   |--_cron_scripts
|   |--AMSTerOrganizer
|   |--diagtoolbox
|   |--optimtoolbox
|   |--TemplatesForPlots
|   |--TSCombiFiles
|   |--zz_Uilities_MT
|   |--zz_Uilities_MT_Ndo
|   |--A lot of scripts...

```

Figure 1 – Directory structure required for the proper usage of AMSTer.

Notes:

- 1) The folders `/$HOME/SAR/AMSTer/AMSTerEngine/_Sources_AE/Older` is used to store the archives and the most recent sources of both components of AMSTerEngine, that is `InSAR` and `MSBASTools`.

Their latest executable version, when compiled, must be moved to `/$HOME/SAR/AMSTer/AMSTerEngine/`.

Hence, when a new version of these packages is available, no need to change the paths of the environmental variables.

- 2) Similarly, the folders `/$HOME/SAR/AMSTer/MSBAS/_3Dsbas_Archives` is used to store the archives and the most recent sources of both components of MSBAS, that is `MSBAS_EXTRACT` and `MSBASi` (where *i* stands for the version). Their latest executable version, when compiled, must be moved to `/$HOME/SAR/AMSTer/MSBAS/`.

Hence, when a new version of these packages is available, no need to change the paths of the environmental variables.

- 3) You can use the script **Check_Installation.sh** to check your installation. See script and manual.

You should also have auxiliary files stored in a folder named “**SAR_AUX_FILES**” in **DataSAR** as shown in **Fig. 2**.

```
$PATH_DataSAR/SAR_AUX_FILES/
|--DEM
|
|--EGM96
|   |--WW15MGH.DAC
|
|--MASKS
|
|--ORBITS
|   |--ENV_ORB
|   |--S1_ORB
```

Figure 2 – Directory structure required for the storage of auxiliary files required by AMSTer Toolbox.

Notes:

- 1) The folders `/$PATH_DataSAR/SAR_AUX_FILES/DEM/` is used to store the DEMs. That is where you store and sort manually your DEMs according to your needs.

It is advised to use another path than the one defined in the state variable `$EXTERNAL_DEMS_DIR`. This last one is used e.g. to create automatically the DEM using the AMSTerEngine command **getSRTMDem** (see manual). However, for the automation with AMSTer Toolbox, to avoid possible confusion, we recommend moving the DEM created in `$EXTERNAL_DEMS_DIR` to

`/$PATH_DataSAR/SAR_AUX_FILES/DEM/`

and indicate that last full path in the `LaunchMTparameters.txt` file for each of your run. If you move the DEM, **remember to change the path** indicated in the `DEM.txt` file accordingly to the new place where the DEM is stored.

The only exception when you may need to have DEMs in `$EXTERNAL_DEMS_DIR` is if you intend to use AMSTerEngine without the AMSTer Toolbox scripts (e.g. using **LazInSAR** command). Indeed, in that case, unless you manually change the configuration text files, it will expect the DEM in that directory.

- 2) The folders `/$PATH_DataSAR/SAR_AUX_FILES/EGM` **must** contain the geoid file in `EGM96/WW15MGH.DAC`.
- 3) The folders `/$PATH_DataSAR/SAR_AUX_FILES/MASKS` is not mandatory, but it is a convenient place to store the mask you would build and used during your processing.

It is advised to use another path than the one defined in the state variable `$EXTERNAL_MASKS_DIR`. This last one is used e.g. to create automatically the water body mask when creating the DEM using the AMSTerEngine command **getSRTMDem** (see manual). However, for the automation with AMSTer Toolbox, to avoid possible confusion, we

recommend moving the masks created in `$EXTERNAL_MASKS_DIR` to `/$PATH_DataSAR/SAR_AUX_FILES/MASKS/` and indicate that last full path in the `LaunchMTparameters.txt` file for each of your run. If you move the mask, **remember to change the path** indicated in the `MaskWB.txt` file accordingly to the new place where the mask is stored.

- 4) The folders `/$PATH_DataSAR/SAR_AUX_FILES/ORBITS` **must** contain the orbits for ENVISAT (if required) or Sentinel 1 (if required). These directories must be associated to the state variable `$ENVISAT_PRECISES_ORBITS_DIR` and `$S1_ORBITS_DIR` respectively.
- 5) To automatically create DEM using the AMSTerEngine command **getSRTMDem** (since AMSTer Engine from October 2024), you need to have the following lines in your `.nertc` (see paragraph 8 below to create a `netrc`):

```
machine e4ftl01.cr.usgs.gov login yourlogin password yourpwd
```

A similar line will be required for downloading Copernicus products from ESA.

5.2 The environmental variables in .bashrc

At the end, your `./bashrc` must look like:

```
# ~/.bashrc: executed by bash(1) for non-login shells.
# see /usr/share/doc/bash/examples/startup-files (in the package bash-doc)
# for examples

# If not running interactively, don't do anything
#case $- in
#    *i*) ;;
#    *) return;;
#esac

[...]

PATH=/opt/local/bin:$PATH
PATH=/usr/bin:$PATH
PATH="/usr/local/bin:/bin:/usr/sbin:/sbin:/snap/bin/"

# trick to avoid error at usage of say function
alias say='echo | espeak -s 120 2>/dev/null'

# AMSTer PATHS
#####

PATH=$PATH:/usr/local/tigervnc/bin/
PATH=$PATH:/home/YourAccount/SAR/EXEC
PATH=$PATH:/home/YourAccount/SAR/AMSTer/MSBAS
PATH=$PATH:/home/YourAccount/SAR/AMSTer/AMSTerEngine
PATH=$PATH:/home/YourAccount/SAR/AMSTer/SCRIPTS_MT
PATH=$PATH:/home/YourAccount/SAR/AMSTer/SCRIPTS_MT/zz_Uutilities_MT_Ndo
PATH=$PATH:/home/YourAccount/SAR/AMSTer/SCRIPTS_MT/zz_Uutilities_MT
PATH=$PATH:/home/YourAccount/SAR/AMSTer/SCRIPTS_MT/_cron_scripts
PATH=$PATH:/home/YourAccount/SAR/AMSTer/SCRIPTS_MT/optimtoolbox
PATH=$PATH:/home/YourAccount/SAR/AMSTer/SCRIPTS_MT/diagtoolbox
PATH=$PATH:/home/YourAccount/SAR/AMSTer/SCRIPTS_MT/AMSTerOrganizer

# AMSTer VARIABLES
#####

export OPENBLAS_NUM_THREADS=1
export JAVA_HOME="/usr/lib/jvm/java-11-openjdk-amd64"
export PATH_HOMEDATA=/mnt/dell3raid5/
export PATH_SynoData=/mnt/syno_data/
export PATH_3602=/mnt/3602/
export PATH_3601=/mnt/3601/
export PATH_3600=/mnt/3600/
export PATH_1650=/mnt/1650/
export PATH_DataSAR=/mnt/syno_sar/
export EARTH_GRAVITATIONAL_MODELS_DIR=${PATH_DataSAR}/SAR_AUX_FILES/EGM
export ENVISAT_PRECISES_ORBITS_DIR=${PATH_DataSAR}/SAR_AUX_FILES/ORBITS/ENV_ORB
export S1_ORBITS_DIR=${PATH_DataSAR}/SAR_AUX_FILES/ORBITS/S1_ORB

export EXTERNAL_DEMS_DIR=/APath/WhereToCreate/DEMS
export EXTERNAL_MASKS_DIR=/APath/WhereToCreate/MASKS

export PATH_SCRIPTS=/home/YourAccount/SAR/AMSTer
export PATHCONV=/usr/bin
export PATHFIJI=/home/YourAccount/SAR/EXEC/Fiji.app/
export PATHGNU=/usr/bin
export PATHTOCPXFIDDLE=/home/YourAccount/SAR/EXEC/
export PATH=$PATH

#export OMP_NUM_THREADS=10,8,4
#export OMP_NUM_THREADS=4,3,2

# Trick to avoid error at usage of say function
alias say='echo | espeak -s 120 2>/dev/null'
```

6) Operating with cron jobs

If you want to operate **AMSTer** from cron jobs, pay attention to the following:

- All variable environments are defined here in the **.bashrc** which is designed for interactive shell. Whatever you do, it won't be sources from scripts if it starts with the following lines (as by default in at least Ubuntu 18.04 and later):


```
# ~/.bashrc: executed by bash(1) for non-login shells.
# see /usr/share/doc/bash/examples/startup-files (in the package bash-doc)
# for examples

# If not running interactively, don't do anything
case $- in
  *i*) ;;
  *) return;;
esac
```
- It is advised to source the **.bashrc** from the **.bash_profile**
- It is advised to define all your state variables at the beginning of your bashrc
- It is advised to launch scripts by redirecting the messages to /dev/null to avoid filling up the mailbox by ending the command line with:


```
> /dev/null 2>&1
```
- You can launch your commands in the crontab as follow (supposing that the path to your bash in /bin):


```
mm hh *** exec /bin/bash -l /path/to/script arg1 arg2 ... > /dev/null 2>&1
```

Example of crontab:

```
# Edit this file to introduce tasks to be run by cron.
# m h dom mon dow  command

SHELL=/bin/bash
BASH_ENV=~/.bashrc

PATH_1650=/mnt/1650
PATH_3600=/mnt/3600
PATH_3601=/mnt/3601
PATH_3602=/mnt/3602
PATH_SynoData=/mnt/syno_data
PATH_DataSAR="/mnt/syno_sar"
PATH_HOMEDATA="/mnt/dell3raid5"
PATH_SCRIPTS=/home/nicolas/SAR/AMSTer

S1_ORBITS_DIR=${PATH_DataSAR}/SAR_AUX_FILES/ORBITS/S1_ORB
ENVISAT_PRECISES_ORBITS_DIR=/mnt/syno_sar/SAR_AUX_FILES/ORBITS/ENV_ORB
EARTH_GRAVITATIONAL_MODELS_DIR=${PATH_DataSAR}/SAR_AUX_FILES/EGM

PATHGNU=/usr/bin
PATHCONV=/usr/bin
PATHFIJI=$HOME/SAR/EXEC/Fiji.app/
PATHTOCPXFIDDLE=${HOME}/SAR/EXEC/

### Domuyo 03h05; 5h35; 17h30
05 03 *** cd /home/nicolas/SAR/AMSTer/SCRIPTS_MT/_cron_scripts/; bash -l -c './Domuyo_S1_Step1_Read_SMCores_Pairs.sh'
35 05 *** cd /home/nicolas/SAR/AMSTer/SCRIPTS_MT/_cron_scripts/; bash -l -c './Domuyo_S1_Step2_MassProc.sh'
30 17 *** cd /home/nicolas/SAR/AMSTer/SCRIPTS_MT/_cron_scripts/; bash -l -c './Domuyo_S1_Step3_MSBAS.sh'
```

7) In case of insufficient RAM

It may happen that your Linux computer runs out of RAM and MSBAS inversion can't be completed. See the document "[Create_Swap_Linux.pdf](#)" which explains how to create a swap file on a Linux computer.

8) Downloading the Sentinel-1 orbits

Processing Sentinel-1 data requires orbits files, either the Near Real Time predicted (PRE) orbits available 180 minutes after the acquisition, or the Non Time Critical Precises (POE) orbits available typically 20 days after the acquisition.

These can be downloaded automatically using the AMSTer Engine function **updateS1Orbits** as a transparent step in the processing, either from ESA or from ASF (Alaska SAR Facility).

In both cases, it requires to have **at least AMSTerEngine V20231213** and a login and a password to be obtained from ESA or ASF, which must be stored in a file named **.netrc** in your home directory. That file must contain one line for either ESA and/or ASF login as follow:

```
machine identity.dataspace.copernicus.eu login <YourLogin> password <YourPassword>
machine urs.earthdata.nasa.gov login <YourLogin> password <YourPassword>
```

The first line is dedicated to the access the ESA server <https://dataspace.copernicus.eu> (this dataspace server replaces the <https://scihub.copernicus.eu> server that was closed in November 2023), while the second line is dedicated to the access the ASF server.

Of course, **<YourLogin>** and **<YourPassword>** must be replaced (without the **< >** symbols) with the login and password obtained from the provider:

- For ESA, visit <https://dataspace.copernicus.eu> and proceed to the registration by following the instructions provided on the web site;
- For ASF, visit urs.earthdata.nasa.gov and proceed to the registration by following the instructions provided on the web site. When registered and logged in, you must also:
 - click on "My Profile", then menu "Applications > Authorized Apps".
 - At the bottom, click on button "APPROVE MORE APPLICATIONS"
 - In the "Search" bar, type "Sentinel"
 - Click on "AUTHORIZE" button next to each application named with Sentinel 1.

Then in a terminal on your computer type:

```
> cd ~
> touch .netrc
> echo "machine urs.earthdata.nasa.gov login <YourLogin> password <YourPassword>" > .netrc
> chmod 0600 .netrc
```

Then

```
> cd ~
> touch .urs_cookies
```

For more information, see: <https://wiki.earthdata.nasa.gov/display/EL/How+To+Access+Data+With+cURL+And+Wget>.

With such a **.netrc** file, providing that your destination directory **S1_ORBITS_DIR** is set up and defined as state variable in your **.bashrc**, you can download the orbits from ESA or ASF by launching either **updateS1Orbits** or **updateS1Orbits -ASF** respectively. This is usually done from the scripts and you do not have to worry about it.

9) Updates in this manual

V 4.1 (October 26 2022):

- add installation of gnu parallel
- add chapter “7) Updates in this manual”

V 4.3 (April 20 2023):

- Refer to last versions of software and scripts
- Check against installer script

V 4.4 (June 27 2023):

- Add reference to technical note on how create swap file

V 5.0 (Aug 30 2023):

- Major changes in dirs and files naming
- Take into account new AMSTer Engine V20230828 which is mandatory

V 6.0 (Oct 25 2023):

- Major changes related to the renaming of MasTer as AMSTer
- Information about public GitHub access to source code and licensing
- Typos and varia

V 6.1 (Dec 15 2023):

- Data server change at ESA in Nov. 2023 requires new .netrc file for allowing the download of Sentinel-1 orbits.

V 6.1.1 (Jan 03 2024):

- Update msbas zip source code V1.2 to avoid compilation error with most recent Mac OSX version

V 6.2.1 (Oct 10 2024):

- Update some python instructions
- Add DEMS and MASKS state variables to cope with new function getSRTMDDEM from AMSTerEngine from October 2024
- Add info about .netrc for SRTM download

V 6.2.2 (Nov 13 2024):

- Information about
msbas_20201009_wExtract_Unified_20220919_Optimized_v1.3_Full3D.zip
- Cosmetic

V 6.2.3 (Aug 07 2025):

- New diagnostic toolbox (by Delphine Smittarello)
- Python packages scikit-gstat and geopandas
- Info about python 3 system-wide installation
- Clang version 20 is mandatory from AMSTerEngine_20250805

V 6.3.0 (Aug 14 2025):

- Info about python 3 installed in virtual environment

- Change QGIS installation

V 6.3.1 (Feb 10 2026):

- Correct path to installer when downloaded from Github Distribution repository

V 7.0 (Aug 20 2026):

- Update license information
- Refer to manual for installation using the installer script
- Update to distribution release from August 20, 2026:
Manual installation procedure checked line by line against AMSTer_install.sh V7.0 (Aug 06 2026) and updated accordingly with AI assistance.